Laboratorio01

October 4, 2026

1 Laboratorio de Práctica

1.1 Nivel Básico

1.1.1 Ejercicio 1: Análisis de una expresión algebraica

Una expresión matemática se recibe como una **cadena** de caracteres. Definir una función `analizar_expresion(expr)` que retorne una **tupla** `(balanceada, n_operadores, variables)` donde:

- `balanceada` es `True` si los paréntesis `( )` y corchetes `[ ]` están correctamente balanceados y anidados, y `False` en caso contrario. (Ayuda: usar una **lista como pila** y un **diccionario** que asocie cada símbolo de cierre con su símbolo de apertura.)
- `n_operadores` es la cantidad de operadores `+ - * / ^` que aparecen en la expresión.
- `variables` es una **lista ordenada** con las letras distintas que aparecen en la expresión (por ejemplo `['x', 'y']`).

Probar con las siguientes expresiones e imprimir el resultado de cada una con un mensaje del tipo:

`(x+1)*(x-1) -> balanceada: True, operadores: 3, variables: ['x']`

[9]:
```
expresiones = ["(x+1)*(x-1)", "[(2*x)+3]/y", "((2*x)+3", "3*x^2-(y+4))",␣
↪"[(x+1])"]
```

[10]:
```
# Solución:
def analizar_expresion(expr):
    parejas = {')': '(', ']': '['}
    operadores = '+-*/^'
    variables = []
    balanceada = True
    n_operadores = 0
    cola = []
    for letra in expr:
        if letra == '[' or letra == '(':
            cola.append(letra)
        elif (letra == ']' or letra == ')') and (len(cola) == 0 or␣
↪parejas[letra] != cola.pop()):
            balanceada = False
        elif letra in operadores:
            n_operadores += 1
```

```
        elif letra.isalpha() and letra not in variables:
            variables.append(letra)
    if len(cola) != 0:
        balanceada = False
    return (balanceada, n_operadores, sorted(variables))
for expresion in expresiones:
    balanceada, n_operadores, variables = analizar_expresion(expresion)
    print('Balanceada:', balanceada, ', operadores:', n_operadores, ',␣
 ↪variables:', variables)
```

```
Balanceada: True , operadores: 3 , variables: ['x']
Balanceada: True , operadores: 3 , variables: ['x', 'y']
Balanceada: False , operadores: 2 , variables: ['x']
Balanceada: False , operadores: 4 , variables: ['x', 'y']
Balanceada: False , operadores: 1 , variables: ['x']
```

1.1.2 Ejercicio 2: Estadística descriptiva de una muestra

Se tiene una muestra de datos en una lista. **Sin usar el módulo `statistics`** para el cálculo:

- Definir `estadisticas(datos)` que retorne una **tupla** `(media, mediana, varianza, desviacion)`, usando la **varianza muestral** (dividiendo entre $n-1$). La mediana debe calcularse **sin modificar** la lista original.
- Con una **lista por comprensión**, crear `z` con los puntajes estandarizados $z_i = (x_i - \bar{x})/s$, redondeados a 4 decimales.
- Crear la lista `atipicos` con los datos cuyo $|z_i| > 2$ (puede usar `zip`).
- Verificar los resultados comparando con `statistics.mean`, `statistics.median` y `statistics.variance` usando `math.isclose`.

Imprimir los resultados y comprobar que `datos` conserva su orden original.

[11]:
```
import math
import statistics

datos = [12, 15, 14, 10, 13, 16, 11, 14, 45, 13]
```

[12]:
```
# Solución:
def estadisticas(datos):
    media = sum(datos)/len(datos)
    length = len(datos)
    datos = sorted(datos)
    varianza = 0
    for i in datos:
        varianza += (i-media)**2
    varianza /= length - 1
    if length%2 == 0:
        mediana = (datos[length//2-1] + datos[length//2])/2
    else:
        mediana = datos[(length-1)//2]
```

```
    return (media, mediana, varianza, math.sqrt(varianza))
media, mediana, varianza, s  =estadisticas(datos)
z = [round((x-media)/s,4) for x in datos]
atipicos = [x for x, z_score in zip(datos, z) if abs(z_score) > 2]

verif_media = math.isclose(media, statistics.mean(datos))
verif_mediana = math.isclose(mediana, statistics.median(datos))
verif_varianza = math.isclose(varianza, statistics.variance(datos))

print(datos)
print(f"Media: {media}")
print(f"Mediana: {mediana}")
print(f"Varianza muestral: {varianza:.4f}")
print(f"Desviación estándar: {s:.4f}")
print(f"Puntajes z: {z}")
print(f"Valores atípicos (|z| > 2): {atipicos}")
print(f"Verificaciones correctas (Media, Mediana, Varianza): {verif_media,␣
 ↪verif_mediana, verif_varianza}")
```

```
[12, 15, 14, 10, 13, 16, 11, 14, 45, 13]
Media: 16.3
Mediana: 13.5
Varianza muestral: 104.9000
Desviación estándar: 10.2421
Puntajes z: [-0.4198, -0.1269, -0.2246, -0.6151, -0.3222, -0.0293, -0.5175,
-0.2246, 2.8022, -0.3222]
Valores atípicos (|z| > 2): [45]
Verificaciones correctas (Media, Mediana, Varianza): (True, True, True)
```

1.1.3 Ejercicio 3: Números complejos como tuplas

Un número complejo $z = a + bi$ se representa como la **tupla** `(a, b)`. Sin usar el tipo `complex` de Python, definir:

- `producto(z, w)` que retorne el producto de dos complejos como tupla. (Desempaquetar: `a, b = z`.)
- `modulo(z)` que retorne $|z|$.
- `a_cadena(z)` que retorne el número como **cadena** con el formato `"5+5i"` o `"3-2i"` (el signo de la parte imaginaria debe escribirse correctamente).
- `potencia(z, n)` que retorne z^n (con $n \geq 0$ entero) multiplicando repetidamente con `producto`.

Pruebas:

1. Imprimir $(1 + 2i)(3 - i)$ en formato cadena.
2. Calcular $(1 + i)^8$ e i^4, e imprimirlos. Verificar con `math.isclose` que $|(1 + i)^8| = |1 + i|^8$.
3. Dada la lista `zs`, imprimir cada número en formato cadena junto con su módulo, **ordenados por módulo creciente**.

[13]:
```
import math

zs = [(1, 2), (3, -1), (0, 1), (-2, -2)]
```

[14]:
```
# Solución:
def producto(z,w):
    a, b = z
    c, d = w
    return a*c - b*d, a*d + b*c
def modulo(z):
    a, b = z
    return math.sqrt(a**2 + b**2)
def a_cadena(z):
    a, b = z
    if b>=0:
        return str(a)+'+'+str(b)+'i'
    else:
        return str(a)+'-'+str(b)+'i'
def potencia(z,n):
    potencia = z
    if n%1 == 0 and n>0:
        for i in range(n-1):
            potencia = producto(potencia,z)
    elif n == 0:
        potencia = (1, 0)
    return potencia
print('(1+2i)(3-i):', a_cadena(producto((1,2),(3,-1))))
print('(1+i)^8:', potencia((1,1),8))
print('Comprobación:', math.isclose(modulo(potencia((1, 1), 8)), (modulo((1,␣
↪1))) ** 8))
zs_ordenados = sorted(zs, key=modulo)
for z in zs_ordenados:
  print(f"Número: {z} -> Módulo: {modulo(z)}")
```

```
(1+2i)(3-i): 5+5i
(1+i)^8: (16, 0)
Comprobación: True
Número: (0, 1) -> Módulo: 1.0
Número: (1, 2) -> Módulo: 2.23606797749979
Número: (-2, -2) -> Módulo: 2.8284271247461903
Número: (3, -1) -> Módulo: 3.1622776601683795
```

1.1.4 Ejercicio 4: Polinomios como diccionarios

Un polinomio se puede representar de forma compacta con un **diccionario** `{exponente: coeficiente}` que guarda solo los términos no nulos. Por ejemplo, $p(x) = 3x^5 - 2x^2 + 7$ es `{5: 3, 2: -2, 0: 7}`.

Definir las siguientes funciones:

- `evaluar(p, x)` — evalúa el polinomio en `x`.
- `derivar(p)` — retorna un **nuevo diccionario** con la derivada de `p` (los términos constantes desaparecen).
- `sumar(p, q)` — retorna un nuevo diccionario con $p + q$, **eliminando** los términos cuyo coeficiente resulte 0. No modificar `p` ni `q`.
- `a_cadena(p)` — retorna el polinomio como **cadena**, con los términos en orden decreciente de exponente. Reglas: el exponente 1 se escribe `x`, el exponente 0 solo el coeficiente, y el coeficiente siempre se escribe (aunque sea 1). Ejemplo: `{4: 15, 1: -4}` → `"15x^4 - 4x"`. El polinomio vacío es `"0"`.

Pruebas con `p = {5: 3, 2: -2, 0: 7}` y `q = {5: -3, 1: 4, 0: -7}`:

1. Imprimir `p`, `p'` y `p + q` en formato cadena.
2. Evaluar `p(2)` y `p'(1)`.
3. Verificar numéricamente que $(p + q)(3) = p(3) + q(3)$.

[15]:
```
p = {5: 3, 2: -2, 0: 7}
q = {5: -3, 1: 4, 0: -7}
```

[16]:
```
# Solución:
def evaluar(p, x):
    eval = 0
    for exponente, coeficiente in p.items():
        eval += coeficiente * x**exponente
    return eval
def derivar(p):
    derivado = {}
    for exponente, coeficiente in p.items():
        if exponente > 0:
            nuevo_coeficiente = coeficiente*exponente
            nuevo_exponente = exponente - 1
            derivado[nuevo_exponente] = nuevo_coeficiente
    return derivado
def sumar(p, q):
    resultado = p.copy()
    for exponente, coeficiente in q.items():
        resultado[exponente] = resultado.get(exponente, 0) + coeficiente
    resultado = {exp: coef for exp, coef in resultado.items() if coef != 0}
    return resultado
def a_cadena(p):
    polinomio = ''
    if len(p) != 0:
        for exponente, coeficiente in p.items():
            polinomio += str(coeficiente)+'x'+'^'+str(exponente)
    else:
        polinomio = '0'
    return polinomio
print('p:', a_cadena(p))
```

```
print("p':", derivar(p))
print('p+q:', sumar(p,q))
print('p(2):', evaluar(p,2))
print("p'(1):", evaluar(derivar(p),2))
print(
    math.isclose(
        evaluar(sumar(p, q), 3), evaluar(p, 3) + evaluar(q, 3)
    )
)
```

```
p: 3x^5-2x^27x^0
p': {4: 15, 1: -4}
p+q: {2: -2, 1: 4}
p(2): 95
p'(1): 232
True
```

1.2 Nivel Intermedio

Comprensiones de listas, tuplas como registros, diccionarios en cálculos, funciones como valores, manejo de errores, módulo `math`.

1.2.1 Ejercicio 5: Aproximaciones del número e

El número e puede aproximarse de dos maneras:

$$S_n = \sum_{k=0}^{n} \frac{1}{k!} \qquad a_n = \left(1 + \frac{1}{n}\right)^n$$

- Definir `sumas_parciales(N)` que retorne la **lista** `[S_0, S_1, ..., S_{N-1}]`.
- Definir `primer_indice(sumas, tol)` que retorne la **tupla** `(n, S_n, error)` del primer índice cuyo error absoluto respecto a `math.e` sea menor que `tol`, o `None` si ninguno lo cumple.
- Con una **lista por comprensión** crear `tabla`, una lista de tuplas `(n, S_n, error)` para `N = 15`, e imprimir las primeras 8 filas con formato alineado.
- Con otra comprensión, crear `limite`, lista de tuplas `(n, a_n, error)` para `n` en `10, 100, 1000, ..., 10^6`, e imprimirla.
- Imprimir `primer_indice(sumas, 1e-6)` y comparar: ¿cuál de las dos aproximaciones converge más rápido?

[17]:
```
# Solución:
def sumas_parciales(N):
    lista = []
    suma = 0
    for k in range(N):
        suma += 1/math.factorial(k)
        lista.append(suma)
    return lista
def primer_indice(sumas, tol):
```

```
    resultado = None
    for i in sumas:
        error = math.fabs(math.e - i)
        if error < tol:
            resultado = (sumas.index(i),i,error)
            break
    return resultado
s_vals = sumas_parciales(15)
tabla = [(n, s, math.fabs(s - math.e)) for n, s in enumerate(s_vals)]
limite = [(10**i, (1+1/i)**i,math.fabs((1+1/i)**i-math.e)) for i in range(1,7)]
print(tabla)
print(limite)
print(primer_indice(s_vals, 1e-6))
```

```
[(0, 1.0, 1.718281828459045), (1, 2.0, 0.7182818284590451), (2, 2.5,
0.2182818284590451), (3, 2.6666666666666665, 0.05161516179237857), (4,
2.708333333333333, 0.009948495125712054), (5, 2.7166666666666663,
0.0016151617923787498), (6, 2.7180555555555554, 0.0002262729034896438), (7,
2.7182539682539684, 2.7860205076724043e-05), (8, 2.71827876984127,
3.0586177750535626e-06), (9, 2.7182815255731922, 3.0288585284310443e-07), (10,
2.7182818011463845, 2.7312660577649694e-08), (11, 2.718281826198493,
2.260552189881082e-09), (12, 2.7182818282861687, 1.7287637987806193e-10), (13,
2.7182818284467594, 1.2285727990501982e-11), (14, 2.71828182845823,
8.149037000748649e-13)]
[(10, 2.0, 0.7182818284590451), (100, 2.25, 0.4682818284590451), (1000,
2.37037037037037, 0.3479114580886753), (10000, 2.44140625, 0.2768755784590451),
(100000, 2.4883199999999994, 0.22996182845904567), (1000000, 2.5216263717421135,
0.19665545671693163)]
(9, 2.7182815255731922, 3.0288585284310443e-07)
```

1.2.2 Ejercicio 6: Variables aleatorias discretas

Una variable aleatoria discreta se representa con un **diccionario** `{valor: probabilidad}`.

- Definir `validar(dist)` que lance un `ValueError` con el mensaje `"Probabilidad negativa"` si alguna probabilidad es negativa, y con el mensaje `"Las probabilidades no suman 1"` si la suma no es 1 (usar `math.isclose`).
- Definir `esperanza(dist)` y `varianza(dist)` que calculen $E[X] = \sum x\,p(x)$ y $\mathrm{Var}(X) = \sum (x-\mu)^2\,p(x)$.
- Definir `suma_independientes(X, Y)` que retorne el diccionario de la distribución de $Z = X + Y$, donde X e Y son independientes: $P(Z=z) = \sum_{x+y=z} P(X=x)P(Y=y)$. (Ayuda: dos bucles anidados sobre `items()` y `get`.)

Pruebas:

1. Validar `X`, `Y` y la distribución incorrecta `malo`, manejando los errores con `try/except` e imprimiendo el mensaje.
2. Calcular `Z = suma_independientes(X, Y)`, imprimirla ordenada por valor (con 4 decimales) y validarla.

3. Verificar con `math.isclose` que $E[Z] = E[X] + E[Y]$ y que $\mathrm{Var}(Z) = \mathrm{Var}(X) + \mathrm{Var}(Y)$.
4. Calcular $P(Z \geq 3)$ con una comprensión sobre el diccionario.

[18]:
```python
import math

X = {0: 0.2, 1: 0.5, 2: 0.3}
Y = {1: 0.4, 3: 0.6}
malo = {0: 0.5, 1: 0.6}
```

[26]:
```python
# Solución:
def validar(dist):
    suma = 0
    for valor, probabilidad in dist.items():
        suma += probabilidad
        if probabilidad < 0:
            raise ValueError("Probabilidad negativa")
    if suma != 1:
        raise ValueError("Las probabilidades no suman 1")
def esperanza(dist):
    esperanza = 0
    for valor, probabilidad in dist.items():
        esperanza += valor*probabilidad
    return  esperanza
def varianza(dist):
    mu = esperanza(dist)
    varianza = 0
    for valor, probabilidad in dist.items():
        varianza += (valor - mu)**2 * probabilidad
    return varianza
def suma_independientes(X,Y):
    Z = {}
    for x, p_x in X.items():
        for y, p_y in Y.items():
            z = x + y
            prob = p_x * p_y
            Z[z] = Z.get(z, 0.0) + prob
    return Z
try:
    validar(X)
    validar(Y)
    validar(malo)
except ValueError as e:
    print(e)
Z = suma_independientes(X,Y)
print(math.isclose(esperanza(Z),esperanza(X)+esperanza(Y)))
print(math.isclose(varianza(Z),varianza(X)+varianza(Y)))
print(sum(prob for valor, prob in Z.items() if valor >= 3))
```

```
Las probabilidades no suman 1
True
True
0.72
```

1.2.3 Ejercicio 7: Método de bisección

Para una función continua f con $f(a)\,f(b) < 0$, el método de bisección aproxima una raíz en $[a, b]$ calculando el punto medio $c = (a + b)/2$ y quedándose con la mitad del intervalo donde f cambia de signo.

Definir `biseccion(f, a, b, tol=1e-6)` que:

- Lance un `ValueError` con el mensaje `"f(a) y f(b) deben tener signos opuestos"` si $f(a)f(b) > 0$.
- Itere hasta que $|f(c)| <$ `tol` o la mitad de la longitud del intervalo sea menor que `tol` (máximo 100 iteraciones).
- Retorne una **tupla** `(raiz, historial)`, donde `historial` es una **lista de tuplas** `(n, a, b, c, f(c))`, una por iteración.

Se proporciona un **diccionario** `problemas` que asocia el nombre de cada ecuación con una tupla `(funcion, a, b)`. Recorrerlo con `.items()` (desempaquetando en el bucle `for`), resolver cada problema dentro de un `try/except` e imprimir:

`x^2 - 2: raíz  1.414214 (21 iteraciones)`

o el mensaje de error si el problema no es válido. Finalmente, imprimir las **primeras 5 filas del historial** del primer problema con formato de columnas alineadas.

[20]:
```python
import math

problemas = {
    "x^2 - 2":     (lambda x: x**2 - 2, 0, 2),
    "cos(x) - x":  (lambda x: math.cos(x) - x, 0, 1),
    "x^3 + x + 1": (lambda x: x**3 + x + 1, -1, 0),
    "x^2 + 1":     (lambda x: x**2 + 1, -1, 1),
}
```

[21]:
```python
# Solución:
def biseccion(f,a,b,tol=1e-6):
    historial = []
    if f(a)*f(b) > 0:
        raise ValueError("f(a) y f(b) deben tener signos opuestos")
    else:
        iteracion = 0
        c = (a+b)/2
        eval = f(c)
        while math.fabs(eval)>tol and (b-a)/2 >tol:
            if f(a)*f(eval) < 0:
                b = c
```

```
            else:
                a = c
            c = (a+b)/2
            eval = f(c)
            iteracion += 1
            historial.append((iteracion, a, b, c, eval))
    return (c, historial)

for funcion, datos in problemas.items():
    try:
        c, historial = biseccion(datos[0], datos[1], datos[2])
        iteraciones = historial[-1][0]
        print(f"{funcion}: raíz  {c:.6f} ({iteraciones} iteraciones)")
    except ValueError as e:
        print(f"{funcion}: {e}")

nombre_func, datos = next(iter(problemas.items()))
_, historial = biseccion(datos[0], datos[1], datos[2])

print(f"\nHistorial para {nombre_func} (Primeras 5 iteraciones):")
print(f"{'Iter':<6} {'a':<10} {'b':<10} {'c':<10} {'f(c)':<12}")
print("-" * 52)

for iteracion, a, b, c, eval_c in historial[:5]:
  print(f"{iteracion:<6} {a:<10.4f} {b:<10.4f} {c:<10.4f} {eval_c:<12.6f}")
```

```
x^2 - 2: raíz  1.500001 (20 iteraciones)
cos(x) - x: raíz  0.750001 (19 iteraciones)
x^3 + x + 1: raíz  -0.915360 (19 iteraciones)
x^2 + 1: f(a) y f(b) deben tener signos opuestos

Historial para x^2 - 2 (Primeras 5 iteraciones):
Iter   a          b          c          f(c)
----------------------------------------------------
1      1.0000     2.0000     1.5000     0.250000
2      1.5000     2.0000     1.7500     1.062500
3      1.5000     1.7500     1.6250     0.640625
4      1.5000     1.6250     1.5625     0.441406
5      1.5000     1.5625     1.5312     0.344727
```

1.3 Nivel Avanzado

Programación orientada a objetos con listas anidadas, tuplas y cadenas; validación y excepciones.

1.3.1 Ejercicio 8: Clase `Matriz`

Diseñar una clase `Matriz` para trabajar con matrices reales, **sin usar `numpy`**. La matriz se almacena como una **lista de listas** (cada sublista es una fila).

La clase debe tener:

- `__init__(self, filas)` — recibe la lista de listas y guarda una **copia** en `self.filas`. Lanza `ValueError("Matriz inválida")` si está vacía o si las filas no tienen todas la misma longitud.
- `forma(self)` — retorna una **tupla** `(num_filas, num_columnas)`.
- `transpuesta(self)` — retorna un **nuevo objeto** `Matriz` con la transpuesta. (Ayuda: `zip(*self.filas)`.)
- `traza(self)` — retorna la suma de la diagonal. Lanza `ValueError("La traza requiere una matriz cuadrada")` si no es cuadrada.
- `__mul__(self, otro)` — producto matricial `self * otro`, retornando un nuevo objeto `Matriz`. Lanza `ValueError` con un mensaje que incluya ambas formas (por ejemplo `"Dimensiones incompatibles: (2, 3) y (2, 3)"`) si el producto no está definido.
- `__eq__(self, otro)` — retorna `True` si ambas matrices tienen las mismas filas.
- `__str__(self)` — retorna una **cadena** con una fila por línea y cada elemento alineado a la derecha en un campo de ancho 6.

Probar con el siguiente código (la salida debe mostrar AB, su traza, y que se cumple $(AB)^T = B^T A^T$; los errores se capturan e imprimen):

```
A = Matriz([[1, 2, 3], [4, 5, 6]])
B = Matriz([[7, 8], [9, 10], [11, 12]])
C = A * B
print(A.forma(), B.forma(), C.forma())
print(C)
print("Traza de C:", C.traza())
print("(AB)^T == B^T A^T:", C.transpuesta() == B.transpuesta() * A.transpuesta())

for intento in [lambda: A * A, lambda: A.traza(), lambda: Matriz([[1, 2], [3]])]:
    try:
        intento()
    except ValueError as e:
        print("Error:", e)
```

[22]:
```
# Solución:
class Matriz():
    def __init__(self, filas):
        if not filas or len(filas[0]) == 0:
            raise ValueError("Matriz inválida")
        length = len(filas[0])
        iguales = True
        for fila in filas:
            if len(fila) != length:
                iguales = False
                break
        if not iguales:
            raise ValueError("Matriz inválida")
        self.filas = filas
    def forma(self):
```

```
        return (len(self.filas), len(self.filas[0]))
    def transpuesta(self):
        nuevas_filas = [list(fila) for fila in zip(*self.filas)]
        return self.__class__(nuevas_filas)
    def traza(self):
        if self.forma()[0] != self.forma()[1]:
            raise ValueError('La transpuesta requiere una matriz cuadrada')
        else:
            suma = 0
            for i in range(self.forma()[0]):
                suma += self.filas[i][i]
            return suma
    def __mul__(self, otro):
        dim1 = self.forma()
        dim2 = otro.forma()
        if dim1[1] != dim2[0]:
            raise ValueError(f'Dimensiones incompatibles:{dim1} y {dim2}')
        else:
            otro_t = otro.transpuesta()
            nuevas_filas = [
                [
                    sum(a * b for a, b in zip(fila_a, columna_b))
                    for columna_b in otro_t.filas
                ]
                for fila_a in self.filas
            ]
            return self.__class__(nuevas_filas)
    def __eq__(self, otro) ->bool:
        if not isinstance(otro, self.__class__):
            return False

        if self.forma() != otro.forma():
            return False

        return all(f1 == f2 for f1, f2 in zip(self.filas, otro.filas))
    def __str__(self) -> str:
        return "\n".join(
          "".join(f"{elemento:>6}" for elemento in fila) for fila in self.filas
      )
```

[23]:
```
A = Matriz([[1, 2, 3], [4, 5, 6]])
B = Matriz([[7, 8], [9, 10], [11, 12]])
C = A * B
print(A.forma(), B.forma(), C.forma())
print(C)
print("Traza de C:", C.traza())
```

```
print("(AB)^T == B^T A^T:", C.transpuesta() == B.transpuesta() * A.
 ↪transpuesta())

for intento in [lambda: A * A, lambda: A.traza(), lambda: Matriz([[1, 2],␣
 ↪[3]])]:
    try:
        intento()
    except ValueError as e:
        print("Error:", e)
```

```
(2, 3) (3, 2) (2, 2)
    58    64
   139   154
Traza de C: 212
(AB)^T == B^T A^T: True
Error: Dimensiones incompatibles:(2, 3) y (2, 3)
Error: La transpuesta requiere una matriz cuadrada
Error: Matriz inválida
 (3, 2) (2, 2)
    58    64
   139   154
Traza de C: 212
(AB)^T == B^T A^T: True
Error: Dimensiones incompatibles:(2, 3) y (2, 3)
Error: La transpuesta requiere una matriz cuadrada
Error: Matriz inválida
```